Ignition Tag Client & AI Mapper Readiness

FactoryLM / MIRA -- Research Assessment

Generated June 15, 2026

Ignition Tag Client & AI-Assisted Tag Mapper — Readiness Assessment

Date: 2026-06-15

Author: Research pass (CHARLIE node)

Branch: `research/ignition-tag-client-readiness`

Scope: How close is MIRA to (a) reading Ignition tags and (b) an AI-assisted tag mapper? Research only — no behavior changes.

Method: Read every Ignition-touching file in the repo (excluding `.claude/worktrees/` copies), executed the relevant test suites, and scored each capability from **verified code**, not from the architecture doc's self-assessment.

■■ ***The architecture doc (`docs/mira-ignition-secure-architecture.md`, dated 2026-05-31) is partly stale.** Its §7/§10 checklist marks D1 (allowlist), D2/D4 (HMAC), and D3 (cloud chat endpoint) as not-built. The code contradicts this — they are built. Every label below is grounded in the code as it exists on `feat/realtime-datapoint-clock` head, with the stale doc markers called out.*

Executive Summary

Can MIRA read Ignition tags today? YES — through a gateway-resident module, which is the intended architecture. MIRA does **not** (and by deliberate design will not) act as a cloud-side OPC-UA/Modbus client reaching into the plant (`ADR-0021`, secure-architecture §8 anti-pattern #1). Instead it ships Jython that runs **inside the customer's Ignition Gateway** and either (a) serves tags over a WebDev GET endpoint, or (b) streams them outbound to the cloud relay. Both code paths exist and the cloud-side ingest is unit-tested green (33/33).

The tag-reading plumbing is code-complete and largely deployed. Folder discovery (`browseTags`), live value reads (`readBlocking` with type/quality/timestamp), a fail-closed allowlist, HMAC signing, the cloud ingest endpoint, the live-value cache, and the per-tag UNS schema are all real. The single material gap is **no verified end-to-end run on the live bench** (PLC → Ignition → cloud → answer), which the architecture doc itself still lists as ■ (item 12).

The AI-assisted tag mapper is mostly design + schema + a prototype, not a working pipeline. The data model is excellent (`tag_entities`, `approved_tags`, `ai_suggestions` type `tag_mapping`), the LLM classifier is fully *specified* (`TagClassification` Pydantic schema), and a **mock connector already does discover → import → normalize → derive-relationships** end-to-end against a fixture. But the production LLM classifier worker (`tag_classifier.py`) does not exist, the connector framework is not wired to any live ingest, and the

Hub tag-import wizard is explicitly "Slice 1 — not built yet."

Bottom line: Reading tags is a *verification + wiring* problem (high readiness). The AI mapper is a *build the classifier + wire the existing pieces* problem (medium-low readiness). The fastest credible demo is the **file-import** → **classify** → **propose** path, which needs no live PLC and reuses the existing `/proposals` surface.

Current State Table

Status legend: **WORKING** (code exists, exercised by passing tests or deployed) · **PARTIAL** (some of it exists/wired) · **PROTOTYPE** (exists but mock/fixture-backed, not wired to prod) · **SPEC'D** (designed, no code) · **MISSING** (absent) · **BENCH-ONLY** (exists but fenced out of the customer product).

Capability Status Evidence (`file:line`)

|---|---|---|---|

1 Read live PLC tag values **directly** (cloud→PLC) **BENCH-ONLY** (forbidden in product) `plc/live-plc-bridge/bridge.py` (direct Modbus TCP poll, BENCH-ONLY banner); `plc/live_monitor.py` (writes GS10). Forbidden for product: `docs/mira-ignition-secure-architecture.md:230` (anti-pattern #1), `.claude/rules/fieldbus-readonly.md`

2 Read tags from Ignition Gateway (gateway-side) **WORKING** `ignition/gateway-scripts/tag-stream.py:99-137` (`_browse_leaf_tags` + `readBlocking`); `ignition/webdev/FactoryLM/api/tags/doGet.py:135,225`

3 Read via WebDev / REST endpoint **WORKING (gateway-resident)** / cloud-pull **MISSING by design** Endpoint: `ignition/webdev/FactoryLM/api/tags/doGet.py:97` (`GET /system/webdev/FactoryLM/api/tags`). No cloud client calls it — relay is push-only: `mira-relay/tag_ingest.py:1-12`

4 Read via MQTT / Sparkplug B **SPEC'D** (prod) / **BENCH-ONLY** (demo) Relay accepts `source_system` incl. nothing for MQTT subscriber; `docs/mira-ignition-secure-architecture.md:178-195` (§6 deferred to `mira-connect`); only MQTT in repo is the fault-detective bench demo: `mira-bridge/flows/fault-detective.json` + `mira-bridge/mosquitto/`

5 Read via OPC UA (MIRA as OPC-UA client) **MISSING by design** No `asyncua`/`opcua` client dep anywhere (only `plc/discover.py:66` lists port 4840 for a read-only scan). Ignition is the OPC-UA client to the PLC, not MIRA — `docs/mira-ignition-secure-architecture.md:19`

6 Discover tag folders / paths **WORKING** `ignition/webdev/FactoryLM/api/tags/doGet.py:135` (`browseTags` + `recurse`), `:181-209` (one-level sub-folder recursion, `MAX_TAGS=500` cap)

7 Read tag metadata (type, quality, ts) **WORKING** `doGet.py:147-152` (name/path/type/`data_type`), `:225-234` (value/quality/timestamp via `readBlocking`)

7b Read tag metadata (**units, descriptions, limits**) **MISSING from live read** (schema ready) Live browse does not read units/description/limits. `tag_entities` schema *has* `units`, `scaling`, `expected_envelope` columns: `mira-hub/db/migrations/025_tag_entities.sql:50-79` — but nothing populates them from a live Ignition browse

8 Import tag export files (JSON/XML/CSV) **PARTIAL / PROTOTYPE** CCW Structured-Text → Ignition tag JSON: `mira-machine-logic-graph/src/ignition/tag-builder.ts`, emits `ignition/tags_micro800.json`. Fixture

JSON tag-export import: `mira-connectors/mira_connectors/mocks/ignition_mock.py:81-91`. No prod CSV/L5X import endpoint wired

9 Normalize tags to a common model **PROTOTYPE** `IgnitionMockConnector.normalize()` → `CanonicalTag` with `data_type`/`engineering_unit`/`proposed_uns_path`: `mira-connectors/mira_connectors/mocks/ignition_mock.py:93-118`. Tested (7/7) but not wired to prod ingest — no `ConnectorService` caller found in repo

10 Classify tag semantics (command/feedback/status/fault/alarm) **SPEC'D** `TagClassification` model (10 categories): `docs/specs/maintenance-namespace-builder-spec.md:597-616`. Worker `mira-crawler/ingest/extractors/tag_classifier.py` **does not exist** (dir has only `__init__.py`, `fault_codes.py`)

11 Group tags into assets **PROTOTYPE** (structural, not semantic) `IgnitionMockConnector.derive_relationships()` emits `HAS_SIGNAL`/`LOCATED_IN` from folder hierarchy + an `_ASSET_FOLDERS` heuristic: `ignition_mock.py:122-160`. Not LLM-driven, not wired to prod

12 Score confidence **PARTIAL** Static in mock (`confidence=0.85`, `ignition_mock.py:113`); `TagClassification.confidence` spec'd (`:613`); `ai_suggestions` carries evidence. No live scoring pipeline

13 Show proposed mappings to a human **PARTIAL** (surface exists, no live producer) `/proposals` renders `ai_suggestions` incl. `type=tag_mapping`: `mira-hub/db/migrations/025_tag_entities.sql:108`, `027_ai_suggestions.sql:34`. Only producer of `tag_mapping` rows today is seed SQL: `tools/seeds/factorylm-garage-conveyor.sql:198-233`. Grouped-by-asset reconciliation table = spec'd golden case #14 (`maintenance-namespace-builder-spec.md:732`), not built

14 Save approved mappings **PARTIAL** (schema + allowlist real; tag_entities writer unwired) Allowlist table: `mira-hub/db/migrations/035_approved_tags.sql`. Semantic catalog w/ approval lifecycle: `025_tag_entities.sql:84-90` (`approval_state`). `ai_suggestions` approval flow exists; no code writes `tag_entities` from an approved `tag_mapping`

15 Use approved mappings in MIRA answers **WORKING (via snapshot)** / enrichment **not wired** `mira-pipeline/ignition_chat.py:133-145,215-231` formats the request's `tag_snapshot` into a grounded prompt preamble (test `test_tag_snapshot_forwarded_as_tag_evidence` passes). Live values persisted in `live_signal_cache` (`mira-relay/tag_ingest.py:452-495`). The engine does **not** yet JOIN `tag_entities` semantic mappings into answers

Test evidence executed this pass

• `mira-relay` ingest pipeline: **33 passed** (`pytest mira-relay/tests -k ingest`).

• `mira-connectors` Ignition mock: **7 passed** (`test_ignition_mock.py`).

• `mira-pipeline` Ignition direct-connection: **4 passed, 1 failed** — `test_no_asset_id_is_plain_chat_turn` expects an educational question ("what is a VFD?") with no `asset_id` to return 200, but the endpoint returns **422 `uns_required`** (`ignition_chat.py:206`). This contradicts the educational carve-out in `.claude/rules/direct-connection-uns-certified.md` ("Educational / general questions … → no gate either way"). **Real discrepancy — either the test is stale or the endpoint over-rejects educational turns.** Tangential to tag-reading, but flagged.

How tag-reading actually works (the mental model)

```
PLANT LAN (never leaves the plant) ■ OUTBOUND 443 ONLY ■ MIRA CLOUD
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ PLC ■■Modbus/OPC-UA■■■ Ignition Gateway ■ ■ ■
■ ■ ■ ■ ■
■ MIRA Jython (in the gateway JVM): ■ ■ ■
■ • doGet.py browseTags + readBlocking ■ ■ ■
■ • tag-stream.py timer: browse→read→filter ■■■HMAC POST /tags/ingest■■
mira-relay/tag_ingest.py
■ allowlist (approved_tags.json) ■ ■ ■ → tag_events (append-only)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ → live_signal_cache
(latest)
■ ■
Perspective "Ask MIRA" ■■■■ chat + tag_snapshot ■■■■HMAC POST /ignition/chat■■
ignition_chat.py → engine
```

Key consequence: "MIRA reads Ignition tags as a client" is true only in the sense that **MIRA's gateway-resident Jython is the client** — running inside Ignition, reading via `system.tag.*`. The cloud is always the *receiver*. There is no cloud-initiated pull, and adding one would violate the security model.

Integration Options, Ranked by MVP Feasibility

1. **Gateway timer-stream → cloud ingest (LIVE)** — *highest plumbing-readiness, but carries the e2e gap.*

Already coded (`tag-stream.py` + `collector.py` + `tag_ingest.py`), HMAC-signed, allowlist-enforced, cloud side green. Needs the **one missing thing**: a verified bench run (PLC → Ignition → relay → cache → answer). Best for a "live data" wow factor; worst for schedule certainty because the unverified link is exactly the demo's spine.

2. **File-import → classify → propose (OFFLINE)** — *fewest unbuilt dependencies; recommended for the first demo.*

Tag list comes from a file, not a live PLC: `mira-machine-logic-graph` already emits a real Ignition tag JSON, and `IgnitionMockConnector` already does discover→import→normalize→derive. Add the spec'd `tag_classifier.py` (cascade + `TagClassification`), emit `ai_suggestions` type `tag_mapping`, render in the **existing** `/proposals` page. No live PLC, no gateway, no network round-trip to verify.

3. **WebDev GET pull from cloud** — *do not build.* Would make the cloud an inbound client to the plant; violates ADR-0021 / anti-pattern #1. Listed only to close it off.

4. **MQTT / Sparkplug subscriber (`mira-connect`)** — *post-MVP.* The non-Ignition edge path. Spec referenced (`sparkplug-uns-bridge-spec.md`) but not drafted; `mira-connect` is deferred. Build after the first Ignition customer.

5. **OPC-UA client in the cloud** — *never (by design).* Same boundary violation as #3.

Readiness Score

Decomposed (scoring from verified code, not the stale doc):

Track Score (0–10) Rationale

|---|---|---|

Read Ignition tags (through-gateway model) **7.5** Browse, read, metadata (type/quality/ts), allowlist (fail-closed), HMAC, cloud ingest, live cache, per-tag UNS schema — all built; ingest tests green; relay deployed in `saas.yml`. **–2.5** only for: no verified live e2e bench run, and units/description/limits not read from the live browse.

AI-assisted tag mapper **3.5** Schema (`tag_entities`/`approved_tags`/`ai_suggestions`) and classifier *spec* are strong; a fixture-backed connector already normalizes + derives relationships and `/proposals` can render `tag_mapping`. **But** the LLM classifier worker is unbuilt, the connector isn't wired to prod, and the Hub import wizard is Slice-1/not-built.

Combined (read + map MVP) **≈5.5 / 10** Reading is a verify-and-wire problem; mapping is a build-the-classifier problem. The pieces are unusually well-shaped — most of the score gap is "assemble," not "invent."

Gap Analysis (what's actually missing)

Reading side (small, well-defined):

• **G-R1 — No verified live end-to-end run.** PLC → Ignition → relay → `live_signal_cache` → Perspective answer has never been proven on the bench (doc item 12, still ■). Everything is unit-tested in isolation; the seam is unproven.

• **G-R2 — Live browse drops rich metadata.** `doGet.py` returns type/quality/ts but not units, descriptions, alarm limits, or scaling, even though `tag_entities` has columns for them. The mapper would want these.

• **G-R3 — `ignition_chat` educational carve-out.** Endpoint returns `422 uns_required` for an ungrounded educational turn; the direct-connection rule says educational questions should pass. One failing test; needs a decision (fix code or fix test).

Mapper side (larger):

• **G-M1 — No LLM classifier.** `mira-crawler/ingest/extractors/tag_classifier.py` is specified but does not exist. This is the core of "AI-assisted."

• **G-M2 — Connector framework not wired.** `mira_connectors` (incl. the Ignition connector) has no production caller — no `ConnectorService` invocation found. It runs only in tests against fixtures.

• **G-M3 — No tag-import UI.** Hub onboarding wizard is "Slice 0" (`company/site/line/finish`); tag-import CSV is "Slice 1 will add" (`onboarding/page.tsx:16`, `api/wizard/[step]/route.ts:28`).

• **G-M4 — No `tag_entities` writer on approval.** Approving a `tag_mapping` proposal has no code path that materializes a `tag_entities` row.

• **G-M5 — No answer-time enrichment.** The engine uses the request's `tag_snapshot`; it does not look up approved semantic mappings (`tag_entities`) to enrich grounding.

Recommended MVP Path

Build the offline file-import mapper first (Option 2). It needs no live PLC and reuses three things that already exist: the connector normalize/derive logic, the `tag_mapping`/`ai_suggestions` schema, and the `/proposals` UI.

1. **Write `tag_classifier.py`** (the spec'd `TagClassification` schema, InferenceRouter cascade, structured JSON mode). Input: a flat tag list (from `machine-logic-graph` export or `IgnitionMockConnector.import_records`). Output: category + candidate component/line/asset + suggested UNS path + confidence. *This is the only genuinely new component.*

2. **Wire it to `ai_suggestions`** as `type=tag_mapping`, one row per tag, with evidence. Reuse the existing proposal-write path (the seed SQL at `tools/seeds/factorylm-garage-conveyor.sql:198` shows the exact row shape).

3. **Render the reconciliation in `/proposals`** grouped by inferred asset (golden case #14). The index and type already exist; this is a UI grouping, not new schema.

4. **On approval, materialize a `tag_entities` row** (close G-M4). Reuse the `kg` approval/promotion pattern.

5. **Then, separately, close the live-read e2e gap (G-R1)** with the bench test the doc already specifies (`tests/e2e/ignition_chat_roundtrip.py`). This upgrades Option 1 from "code-complete" to "demonstrated."

This sequence makes the AI mapper demonstrable in days (it's one new worker + wiring), and de-risks the live path on its own timeline rather than as a demo dependency.

"Do Not Build Yet" List

• ■ **A cloud-side OPC-UA / Modbus / EtherNet-IP client.** Boundary violation (ADR-0021, anti-pattern #1). MIRA reads *through* Ignition, never from the cloud into the plant.

• ■ **A cloud puller that calls the WebDev `/api/tags` GET.** Same violation — the cloud must not initiate into the plant. The endpoint is for the gateway/Perspective, not the cloud.

• ■ **The MQTT/Sparkplug subscriber (`mira-connect`).** Post-MVP. Don't draft `sparkplug-uns-bridge-spec.md` or build the subscriber until there's an Ignition customer live.

• ■ **Any tag-write path.** Read-only is the product. Writes "don't exist in the customer-shipped story" (`.claude/rules/fieldbus-readonly.md`).

• ■ **Promoting the fault-detective bench compose (mosquitto + live-plc-bridge) to a customer architecture.** It implies MIRA hosts the broker and polls the PLC — the exact anti-pattern (#3/#4). Bench harness only.

• ■ **A full Ignition Java/Kotlin Module (JAR).** The Perspective + WebDev + gateway-script bundle covers the MVP; the packaged Module is post-MVP Exchange polish.

• ■ **Over-engineering metadata extraction (G-R2) before the classifier exists.** Units/limits are nice-to-have; the classifier (G-M1) is the wedge.

Source Index (files read this pass)

• `mira-relay/tag_ingest.py`, `mira-relay/relay_server.py`, `mira-relay/clock_resolver.py`

• `mira-pipeline/ignition_chat.py` (+ `tests/test_ignition_chat_direct_connection.py`)

• `ignition/webdev/FactoryLM/api/tags/{doGet.py,collector.py,allowlist.py}`, `ignition/gateway-scripts/tag-stream.py`, `ignition/README.md`

• `mira-connectors/mira_connectors/mocks/ignition_mock.py` (+ `tests/test_ignition_mock.py`)

• `mira-machine-logic-graph/` (README + `src/ignition/`)

• `mira-hub/db/migrations/{025_tag_entities,027_ai_suggestions,033_tag_events,035_approved_tags,036_current_tag_state_freshness}.sql`

• `mira-hub/src/app/(hub)/onboarding/page.tsx`, `mira-hub/src/app/api/wizard/[step]/route.ts`

• `docs/mira-ignition-secure-architecture.md`, `docs/specs/maintenance-namespace-builder-spec.md` (§AI Pipeline)

• `docker-compose.saas.yml` (relay + HMAC), `mira-bridge/` (fault-detective MQTT bench only)

• `.claude/rules/{direct-connection-uns-certified,fieldbus-readonly}.md`